Technical Note: `scikit-cuda` as a Dependency of `cuvarbase`

cuvarbase fork (`WilliamCFong/cuvarbase`)

April 28, 2026

Abstract

`cuvarbase` is a PyCUDA-backed library of GPU period-finding tools for astronomical time series. It depends on `scikit-cuda` for one thing only: GPU FFT plans and the inverse-FFT call inside the adjoint NFFT used by the Lomb–Scargle periodogram. `scikit-cuda` has not had a release since June 2019 and is, in practice, abandoned. This note documents (a) exactly how `cuvarbase` consumes `scikit-cuda`, (b) the concrete failures caused by upstream's lack of maintenance against modern NumPy and Setuptools, (c) the workaround currently applied in this fork, and (d) a recommended migration path.

1 Background

1.1 What `cuvarbase` is

`cuvarbase` implements several time-series period finders on the GPU: generalized Lomb–Scargle (GLS), Box-Least-Squares (BLS), Conditional Entropy (CE), Phase-Dispersion Minimization (PDM2), and an adjoint Non-equispaced FFT (NFFT) that the GLS path consumes. Kernels live in `src/cuvarbase/kernels/*.cu` and are compiled at runtime via PyCUDA's `SourceModule`. PyCUDA itself supplies device memory, streams, and the JIT compiler — but *not* an FFT.

1.2 What `scikit-cuda` is

`scikit-cuda` (`lebedov/scikit-cuda`) is a Python wrapper around NVIDIA's CUDA math libraries (cuBLAS, cuFFT, cuSOLVER, cuRAND, etc.). For the FFT in particular, `skcuda.fft` wraps cuFFT plans and execution functions so callers can stay in Python and exchange PyCUDA `GPUArrays`.

1.3 Why `cuvarbase` pulls it in

The adjoint NFFT in `src/cuvarbase/cunfft.py` needs an inverse FFT on a gridded, oversampled spectrum after the kernel-spreading step. Writing a cuFFT binding by hand was avoided in 2017 by reusing `skcuda.fft.Plan`/`ifft`.

2 How `cuvarbase` actually uses `scikit-cuda`

The surface area is small. Across the entire package and test suite there are exactly two call sites for two APIs:

Listing 1: `src/cuvarbase/cunfft.py`

```
import skcuda.fft as cufft
# ...
self.cu_plan = cufft.Plan(self.n, self.complex_type,
    self.complex_type,
                              stream=self.stream)
```

```
# ...
cufft.ifft(memory.ghat_g, memory.ghat_g, memory.cu_plan)
```

Listing 2: `tests/test_nfft.py`

```
import skcuda.fft as cufft
# ...
plan = cufft.Plan(len(y), np.complex128, np.complex128)
cufft.ifft(yg, yghat, plan)
```

That is the entire dependency: a constructor (`Plan`) and an in-place inverse-FFT call (`ifft`). No cuBLAS, cuSOLVER, cuRAND, or other `skcuda.*` subpackage is touched. Everything else is unaffected.

3 Maintenance status of `scikit-cuda`

- **Last PyPI release:** 0.5.3 (May 2019). No release in the intervening ∼6 years (as of April 28, 2026).
- **Upstream activity:** the GitHub repository `lebedov/scikit-cuda` accumulates issues and PRs without merges. It is, by any reasonable definition, archived in spirit if not in flag.
- **Wheel availability:** two `py2.py3-none-any` wheels exist on PyPI for 0.5.2/0.5.3 plus three sdists. The pure-Python wheels install cleanly; the runtime requirements (CUDA, PyCUDA, recent NumPy) do not.

4 Concrete breakage caused by lack of maintenance

4.1 NumPy 1.25 removed `np.typeDict`

`skcuda/misc.py:637` contains:

```
num_types = [np.typeDict[t] for t in
             ('float32', 'float64', 'complex64', 'complex128')]
```

`numpy.typeDict` was deprecated in NumPy 1.20 and removed in NumPy 1.25. Any `import skcuda.fft` (or any `from skcuda import *`) under NumPy $\geq$ 1.25 raises:

```
AttributeError: module 'numpy' has no attribute 'typeDict'
```

On a fresh `nox -s tests` install with no NumPy pin, pip resolves to NumPy 2.x and both `tests/test_lombscargle.py` and `tests/test_nfft.py` fail at *collection time* — the test files cannot even be imported.

4.2 `pkg_resources` deprecation

`skcuda/__init__.py` declares its namespace via `pkg_resources.declare_namespace`. Setuptools is in the process of removing `pkg_resources` (slated for as early as 2025-11-30 per the deprecation warning emitted on every import). Once the API is gone, `import skcuda` fails outright. The project is currently emitting:

```
UserWarning: pkg_resources is deprecated as an API.
DeprecationWarning: Deprecated call to
    pkg_resources.declare_namespace('skcuda').
```

4.3 PyCUDA version drift

`cuvarbase`'s pin already excludes `pycuda == 2024.1.2` (a known-bad release), but scikit-cuda has not been validated against any PyCUDA released in the last several years. Future PyCUDA API changes carry no guarantee of an upstream skcuda fix.

5 Current workaround in this fork

PR #2 caps NumPy below the breaking version in `pyproject.toml`:

Listing 3: `pyproject.toml`

```
dependencies = [
    # scikit-cuda 0.5.3 still uses `np.typeDict`
        (skcuda/misc.py:637),
    # which was removed in NumPy 1.25. Cap numpy until skcuda
        releases
    # a fix.
    "numpy>=1.6,<1.25",
    "scipy",
    "pycuda>=2017.1.1,!=2024.1.2",
    "scikit-cuda",
]
```

This restores test collection. It is a stop-gap with two costs:

1. NumPy 1.24 is the last version under the cap. Subsequent NumPy improvements (performance, new APIs, native macOS arm64 support, NumPy 2 typing) are inaccessible to consumers of `cuvarbase`.

2. NumPy 1.24 has no Python 3.12 wheels, so the practical Python upper bound is 3.11 even though `requires-python` still says `>=3.9` and the classifiers list 3.12.

   The pin does *nothing* for the `pkg_resources` timebomb in section §4.2.

6 Recommended migration

The skcuda surface area in `cuvarbase` is small enough — one `Plan` object, one `ifft` call — that direct replacement is straightforward. Three viable replacements, in rough order of fit:

`cuda-python` (`nvidia-cuda-python`)
: NVIDIA's official low-level Python bindings. Includes cuFFT bindings via `cuda.bindings.cufft`. Supported, versioned with the CUDA toolkit, and the canonical answer going forward. Requires writing a thin wrapper (plan-handle creation, exec call, destroy) but the control flow is a direct map from the existing two call sites.

`nvmath-python`
: NVIDIA's high-level math-libraries package (cuFFT, cuBLAS, cuSPARSE, cuSOLVER) layered on top of `cuda-python`. Closer ergonomically to `skcuda.fft`, but newer and dependency-heavy.

`cupy`
: `cupy.fft` wraps cuFFT and integrates with PyCUDA arrays via the `__cuda_array_interface__` protocol. Pulls in CuPy as a dependency, which is a sizeable addition for what amounts to two function calls.

Estimated effort

- `src/cuvarbase/cunfft.py`: replace the `Plan` construction in `NFFTMemory.allocate_grid` and the `cufft.ifft` call in `nfft_adjoint_async`.
- `tests/test_nfft.py`: replace the equivalent two lines used in the CPU-vs-GPU comparison fixture.
- Update `pyproject.toml`: drop `scikit-cuda`, drop the `numpy<1.25` cap, add the chosen replacement.
- Update CLAUDE.md and `INSTALL.rst`: scikit-cuda is no longer a runtime dependency.

A reasonable bound: a single afternoon of focused work plus a re-run of the GPU test suite. The migration is a strict removal of risk; nothing is lost.

7 Action items

1. **Short term:** keep the `numpy<1.25` cap from PR #2 in place. Document it (this note).
2. **Medium term:** prototype the `cuda-python` cuFFT path in a feature branch, validated against `tests/test_nfft.py`.
3. **Long term:** delete `scikit-cuda` from `dependencies`, lift the NumPy cap, restore Python 3.12 to the supported matrix, and remove this note.